Section 4: Strings

This section is part of the 5-hour LeetCode DSA Foundations course by Mukul Raina

- YouTube Course: https://youtu.be/YJMBv4Q2Tg8?si=Q1vDzusUJf753GG3
- YouTube Channel: https://www.youtube.com/@mukulraina
- Instructor LinkedIn: https://www.linkedin.com/in/mukulraina/

C11: DSA Foundations for Interviews

Section 4: Strings

- SLIDE 31: Section Intro, Strings
- SLIDE 32: Java String Fundamentals: Immutability, StringBuilder, and .equals()
- SLIDE 33: Palindrome Patterns: Two-Pointer Check
- SLIDE 33B: Expand Around Center: Finding Palindromic Substrings
- SLIDE 34: Anagrams and Character Frequency
- SLIDE 35: Character Frequency Maps: Array vs HashMap
- SLIDE 36: Worked Example: Longest Substring Without Repeating Characters
- SLIDE 37: Wrong vs Right: String Mistakes
- SLIDE 38: Interview Practice: Strings
- SLIDE 39: Section Cheat Sheet

C11: DSA Foundations for Interviews

Section 4: Strings

SLIDE 31: Section Intro, Strings

Section 4: Strings

String problems are the second most common category in coding interviews, behind arrays. They appear at every level from phone screens to onsite rounds, and companies like Google, Amazon, and Microsoft draw heavily from the string problem pool for fresh graduate hiring.

The good news: if you understood Section 3 (Arrays) well, you already know roughly half of what you need for strings. A Java String is backed by a character array, so techniques like two pointers, frequency counting, and sliding windows apply directly. This section teaches the other half: Java-specific string behavior that creates hidden performance traps, palindrome and anagram patterns that appear repeatedly, and the character frequency toolkit that underpins dozens of problems.

What We'll Cover:

1. Java string internals: immutability, StringBuilder, and the .equals() pitfall
2. Palindrome detection with two pointers and expand-around-center

3. Anagram detection and grouping with character frequency
4. Choosing between int[26], int[128], and HashMap for frequency maps
5. Worked example: Longest Substring Without Repeating Characters
6. Common string mistakes and how to avoid them

Where Strings Connect to Other Sections:

Connection	Section	How It Applies
Character frequency maps are hash maps applied to characters	Section 5: Hash Maps	Same lookup optimization, character-scoped
Two-pointer palindrome check uses the opposite-direction pattern	Section 6: Two Pointers	Left and right pointers moving inward
"Longest/shortest substring with condition" is a sliding window	Section 7: Sliding Window	Variable-size window on a character array
Subsequence problems (not substring) require DP	C6: Trees, Graphs, DP	Different course, different technique

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 32: Java String Fundamentals: Immutability, StringBuilder, and .equals()

Strings in Java Are Immutable

Every time you "modify" a String in Java, the runtime creates an entirely new String object. The original is untouched. This has a direct impact on time complexity that interviewers expect you to understand.

The Hidden O(n^2) Trap: String Concatenation in a Loop

```
// Bad: O(n^2) total time
String result = "";
for (int i = 0; i < n; i++) {
    result += chars[i];   // creates a new String of length i+1 each iteration
}
// Total characters copied: 1 + 2 + 3 + ... + n = n*(n+1)/2 = O(n^2)
```

```
// Good: O(n) total time
StringBuilder sb = new StringBuilder();
for (int i = 0; i < n; i++) {
    sb.append(chars[i]);  // amortized O(1) per append
}
String result = sb.toString();
```

Each += iteration copies the entire existing string into a new object and appends one character. After n iterations, the total characters copied is 1 + 2 + 3 + ... + n = n(n+1)/2, which is O(n^2). StringBuilder maintains a resizable internal buffer and appends in amortized O(1), making the loop O(n).

The .equals() Pitfall

```java
String a = new String("hello");
String b = new String("hello");

System.out.println(a == b);       // false (compares references)
System.out.println(a.equals(b));  // true  (compares content)
```

The == operator checks if two references point to the same object in memory. The .equals() method checks if the character sequences are identical. In interviews, using == for string comparison is a red flag that tells the interviewer you do not understand Java fundamentals.

Core String Operations and Their Costs:

Operation	Method	Time Complexity	Notes
Access character	charAt(i)	O(1)	Direct array index
Length	length()	O(1)	Stored field
Substring	substring(i, j)	O(j - i)	Creates new String
Concatenation	s1 + s2	O(n + m)	Creates new String
Search	indexOf(target)	O(n * m)	Naive substring search
Comparison	equals(other)	O(n)	Character by character
To char array	toCharArray()	O(n)	Full copy
StringBuilder append	append(c)	Amortized O(1)	Resizable buffer

Interview Tip: If you write string concatenation inside a loop during an interview, the interviewer may ask about the time complexity. Know that += is O(n^2) total and be ready to switch to StringBuilder. This is one of the most common Java-specific follow-up questions.

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 33: Palindrome Patterns: Two-Pointer Check

What Is a Palindrome?

A palindrome reads the same forward and backward. “racecar” is a palindrome. “hello” is not. Palindrome problems are among the most frequently asked string questions in interviews because they test two-pointer reasoning, edge case handling, and sometimes expand-around-center logic.

Two-Pointer Palindrome Check

Place one pointer at the start and one at the end. Compare characters and move inward. If every pair matches, the string is a palindrome.

```java
public boolean isPalindrome(String s) {
    int left = 0;
    int right = s.length() - 1;

    while (left < right) {
        if (s.charAt(left) != s.charAt(right)) {
```

```
            return false;
        }
        left++;
        right--;
    }
    return true;
}
```

Time O(n), space O(1). This is the same opposite-direction two-pointer pattern from Section 6, applied to characters instead of numbers.

Worked Example: Valid Palindrome (LeetCode 125)

The problem adds a twist: consider only alphanumeric characters, ignore case.

"A man, a plan, a canal: Panama" is a valid palindrome after filtering.

```
public boolean isPalindrome(String s) {
    int left = 0;
    int right = s.length() - 1;

    while (left < right) {
        while (left < right && !Character.isLetterOrDigit(s.charAt(left))) {
            left++;
        }
        while (left < right && !Character.isLetterOrDigit(s.charAt(right))) {
            right--;
        }
        if (Character.toLowerCase(s.charAt(left)) != Character.toLowerCase(s.charAt
(right))) {
            return false;
        }
        left++;
        right--;
    }
    return true;
}
```

The inner while loops skip non-alphanumeric characters. The comparison uses toLowerCase to normalize case. Time O(n), space O(1).

Step-by-Step Trace: "A man, a plan, a canal: Panama"

Step	Left Char	Right Char	Match?	Action
1	'A'	'a'	Yes (case-insensitive)	Move inward
2	'm'	'm'	Yes	Move inward
3	'a'	'a'	Yes	Move inward
4	'n'	'n'	Yes	Move inward
...	...	...	Yes	Continue
Final	Pointers cross			Return true

SLIDE 33B: Expand Around Center: Finding Palindromic Substrings

When the Question Changes from "Is It a Palindrome?" to "Find Palindromes Within It"

The two-pointer check from Slide 33 answers a yes/no question about the entire string. But problems like "Longest Palindromic Substring" (LeetCode 5) ask you to find palindromic substrings within a larger string. This requires a different technique: expand around center.

The Expand-Around-Center Technique

For each possible center position, expand outward as long as characters on both sides match. A palindrome can be centered on a single character (odd length: "aba") or between two characters (even length: "abba"). So for a string of length n, there are 2n - 1 possible centers.

```
public String longestPalindrome(String s) {
    if (s == null || s.length() < 1) return "";

    int start = 0;
    int maxLen = 0;

    for (int i = 0; i < s.length(); i++) {
        int len1 = expandAroundCenter(s, i, i);       // odd-length palindromes
        int len2 = expandAroundCenter(s, i, i + 1);   // even-length palindromes
        int len = Math.max(len1, len2);

        if (len > maxLen) {
            maxLen = len;
            start = i - (len - 1) / 2;
        }
    }
    return s.substring(start, start + maxLen);
}

private int expandAroundCenter(String s, int left, int right) {
    while (left >= 0 && right < s.length() && s.charAt(left) == s.charAt(right)) {
        left--;
        right++;
    }
    return right - left - 1;
}
```

Time O(n^2): n centers, each expansion up to O(n). Space O(1): only pointer variables.

Trace: "babad"

Center	Type	Expansion	Palindrome Found	Length
i=0	Odd	“b”	“b”	1
i=0	Even	“ba”	No match	0
i=1	Odd	“a” then “bab”	“bab”	3
i=1	Even	“ab”	No match	0
i=2	Odd	“b” then “aba”	“aba”	3
i=2	Even	“ba”	No match	0
i=3	Odd	“a”	“a”	1
i=3	Even	“ad”	No match	0
i=4	Odd	“d”	“d”	1

Longest palindromic substring: “bab” or “aba” (both length 3). The algorithm returns whichever it finds first.

Palindrome Pattern Decision:

Problem Type	Technique	Time	Space
“Is this string a palindrome?”	Two pointers (opposite direction)	O(n)	O(1)
“Find longest palindromic substring”	Expand around center	O(n^2)	O(1)
“Count palindromic substrings”	Expand around center, count instead of track max	O(n^2)	O(1)
“Longest palindromic subsequence”	Dynamic Programming (C6)	O(n^2)	O(n^2)

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 34: Anagrams and Character Frequency

What Is an Anagram?

An anagram is a rearrangement of all characters of one string to form another. “listen” and “silent” are anagrams. Same characters, same counts, different order.

Two Approaches to Check Anagrams

Approach 1: Sort and Compare

Sort both strings and check if the sorted results are equal. If two strings are anagrams, their sorted forms are identical.

```
public boolean isAnagram(String s, String t) {
    if (s.length() != t.length()) return false;

    char[] sArr = s.toCharArray();
    char[] tArr = t.toCharArray();
    Arrays.sort(sArr);
    Arrays.sort(tArr);
    return Arrays.equals(sArr, tArr);
}
```

Time O(n log n) due to sorting. Space O(n) for the character arrays.

Approach 2: Frequency Count with int[26]

Count character frequencies for the first string, then decrement for the second. If all counts are zero at the end, the strings are anagrams.

```
public boolean isAnagram(String s, String t) {
    if (s.length() != t.length()) return false;

    int[] freq = new int[26];
    for (int i = 0; i < s.length(); i++) {
        freq[s.charAt(i) - 'a']++;
        freq[t.charAt(i) - 'a']--;
    }

    for (int count : freq) {
        if (count != 0) return false;
    }
    return true;
}
```

Time O(n), space O(1) (the int[26] array is fixed size regardless of input). This is the preferred approach in interviews for lowercase English letter inputs.

The Increment/Decrement Pattern: Build up counts with one string, tear them down with the other, check that everything balances to zero. You will see this pattern again in sliding window substring problems (Section 7) and in frequency matching problems throughout your interview preparation.

Group Anagrams (LeetCode 49)

Group a list of strings so that anagrams are together. The key insight: all anagrams produce the same string when sorted. Use the sorted string as a HashMap key.

```
public List<List<String>> groupAnagrams(String[] strs) {
    Map<String, List<String>> map = new HashMap<>();

    for (String s : strs) {
        char[] chars = s.toCharArray();
        Arrays.sort(chars);
        String key = new String(chars);
        map.computeIfAbsent(key, k -> new ArrayList<>()).add(s);
    }
    return new ArrayList<>(map.values());
}
```

Time O(n * k log k) where n is the number of strings and k is the maximum string length. The computeIfAbsent method is the clean Java idiom for “get the list for this key, or create one if it doesn’t exist yet.”

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

SLIDE 35: Character Frequency Maps: Array vs HashMap

Frequency counting is the backbone of string pattern problems. The tool you choose depends on the character set the problem specifies.

Option 1: int[26] for Lowercase English Letters

```
int[] freq = new int[26];
for (char c : s.toCharArray()) {
    freq[c - 'a']++;
}
```

The expression c - ‘a’ converts a character to an index: ‘a’ becomes 0, ‘b’ becomes 1, through ‘z’ becoming 25. This works because Java characters have contiguous ASCII values.

Option 2: int[128] for Full ASCII

```
int[] freq = new int[128];
for (char c : s.toCharArray()) {
    freq[c]++;
}
```

Covers uppercase, lowercase, digits, punctuation, and control characters. Use when the problem says “any ASCII character” or does not specify a restricted character set.

Option 3: HashMap<Character, Integer> for Unknown or Unicode Character Sets

```
Map<Character, Integer> freq = new HashMap<>();
for (char c : s.toCharArray()) {
    freq.put(c, freq.getOrDefault(c, 0) + 1);
}
```

No size assumption. Works for any character set including Unicode. Slightly slower due to boxing and hashing overhead, but necessary when the character set is unbounded.

Decision Table:

Character Set	Tool	Space	Speed	When to Use
Lowercase English (a-z)	int[26]	O(1), 26 ints	Fastest, direct array access	Problem says “lowercase English letters”
ASCII	int[128]	O(1), 128 ints	Fast, direct array access	Problem says “ASCII characters” or “printable characters”
Unicode or unspecified	HashMap	O(k), k = distinct chars	Slower, hash overhead	Problem says “any character” or does not specify
Known small alphabet	int[size]	O(1)	Fast	Digits (int[10]), hex (int[16])

The Increment/Decrement Pattern in Detail

This single pattern solves anagram checks, frequency matching, and sliding window character tracking:

```java
int[] freq = new int[26];

// Build frequency from string s
for (char c : s.toCharArray()) {
    freq[c - 'a']++;
}

// Tear down frequency with string t
for (char c : t.toCharArray()) {
    freq[c - 'a']--;
}

// Check: all zeros means exact match
boolean match = true;
for (int count : freq) {
    if (count != 0) {
        match = false;
        break;
    }
}
```

Interview Tip: When the problem says “lowercase English letters,” use int[26] without hesitation. It is faster, uses less memory, and signals to the interviewer that you understand the constraint. Using a HashMap when int[26] suffices is not wrong, but it is a missed opportunity to show precision.

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 36: Worked Example: Longest Substring Without Repeating Characters

Problem (LeetCode 3): Given a string s, find the length of the longest substring without repeating characters.

Step 1: Understand

A substring is contiguous (unlike a subsequence which can skip). “Repeating” means any character appears more than once within the window. We need the maximum length of such a window.

Input: “abcabcbb”. Output: 3 (the substring “abc”).

Step 2: Identify the Pattern

“Longest substring with a condition” is the recognition signal for a variable-size sliding window (Section 7 preview). The condition is “all characters unique.” We expand the window rightward, and when a duplicate enters, we shrink from the left until the duplicate is removed.

Step 3: Plan

Use a HashSet to track characters in the current window. Right pointer expands. When s.charAt(right) is already in the set, remove s.charAt(left) from the set and advance left. Track maximum window size.

Step 4: Implement

```java
public int lengthOfLongestSubstring(String s) {
    Set<Character> window = new HashSet<>();
    int left = 0;
    int maxLen = 0;

    for (int right = 0; right < s.length(); right++) {
        while (window.contains(s.charAt(right))) {
            window.remove(s.charAt(left));
            left++;
        }
        window.add(s.charAt(right));
        maxLen = Math.max(maxLen, right - left + 1);
    }
    return maxLen;
}
```

Step 5: Verify

Trace through "abcabcbb":

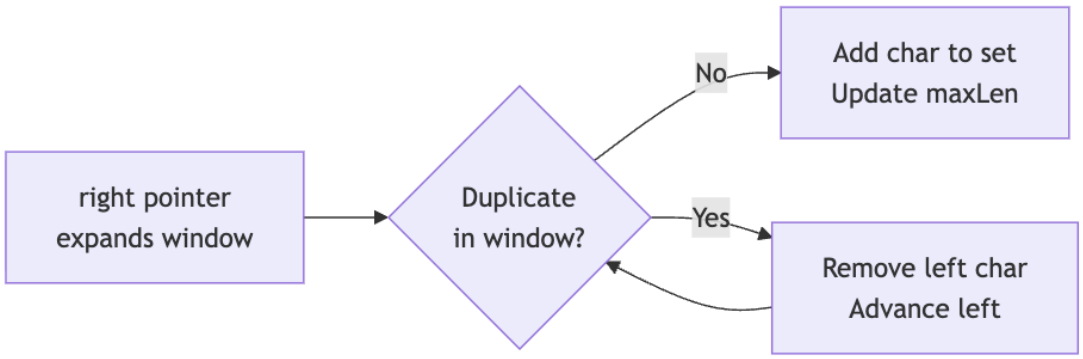
right	char	Window Before	Action	Window After	left	maxLen
0	'a'	{}	Add 'a'	{a}	0	1
1	'b'	{a}	Add 'b'	{a,b}	0	2
2	'c'	{a,b}	Add 'c'	{a,b,c}	0	3
3	'a'	{a,b,c}	'a' in set: remove 'a', left=1	{b,c}	1	3
		{b,c}	Add 'a'	{b,c,a}	1	3
4	'b'	{b,c,a}	'b' in set: remove 'b', left=2	{c,a}	2	3
		{c,a}	Add 'b'	{c,a,b}	2	3
5	'c'	{c,a,b}	'c' in set: remove 'c', left=3	{a,b}	3	3
		{a,b}	Add 'c'	{a,b,c}	3	3
6	'b'	{a,b,c}	'b' in set: remove 'a', left=4	{b,c}	4	3
		{b,c}	'b' in set: remove 'b', left=5	{c}	5	3
		{c}	Add 'b'	{c,b}	5	3
7	'b'	{c,b}	'b' in set: remove 'c', left=6	{b}	6	3

right	char	Window Before	Action	Window After	left	maxLen
		{b}	'b' in set: remove 'b', left=7	{}	7	3
		{}	Add 'b'	{b}	7	3

Result: 3. The longest substring without repeating characters is "abc".

Complexity: Time O(n). Each character is added to the set once and removed at most once, so the total operations across both pointers is at most 2n. Space O(min(n, charset size)), since the set holds at most one of each distinct character.

Why This Problem Matters: It combines string traversal, HashSet usage, and the sliding window pattern. It will be revisited formally in Section 7 as a sliding window application using the variable-size longest template.



Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

SLIDE 37: Wrong vs Right: String Mistakes

Mistake 1: Comparing Strings with == Instead of .equals()

Wrong:

```
if (s1 == s2) { ... }
```

This compares memory addresses, not string content. Two strings with identical characters created via new String() will return false with ==.

Right:

```
if (s1.equals(s2)) { ... }
```

Always use .equals() for content comparison. Use equalsIgnoreCase() when case does not matter.

Mistake 2: String Concatenation in a Loop

Wrong:

```
String result = "";
for (char c : chars) {
    result += c;    // O(n^2) total: copies entire string each iteration
}
```

Right:

```
StringBuilder sb = new StringBuilder();
for (char c : chars) {
    sb.append(c);   // Amortized O(1) per append, O(n) total
}
String result = sb.toString();
```

Mistake 3: Using HashMap When int[26] Suffices

Wrong:

```
// Problem says "lowercase English letters"
Map<Character, Integer> freq = new HashMap<>();
for (char c : s.toCharArray()) {
    freq.put(c, freq.getOrDefault(c, 0) + 1);
}
```

This works but introduces unnecessary boxing overhead, hash computation, and more verbose code when the character set is fixed and small.

Right:

```
int[] freq = new int[26];
for (char c : s.toCharArray()) {
    freq[c - 'a']++;
}
```

Faster, cleaner, and signals constraint awareness to the interviewer.

Mistake 4: Confusing Substring with Subsequence

Wrong assumption: treating a “subsequence” problem with contiguous window techniques.

- **Substring:** contiguous characters. “abc” is a substring of “xabcy”. Solved with sliding window, two pointers, prefix techniques.
- **Subsequence:** characters in order but can skip. “ace” is a subsequence of “abcde”. Solved with dynamic programming (C6).

Right approach: when the problem says “subsequence,” stop and ask yourself whether contiguous techniques apply. They almost never do. Subsequence problems typically require DP.

Mistake 5: Forgetting Edge Cases

Common missed cases for string problems:

Edge Case	What Breaks	How to Handle
Empty string “”	Length-dependent logic, charAt on index 0	Check s.length() == 0 early
Single character “a”	Palindrome checks, two-pointer initialization	Typically return true or the character itself
All same characters “aaaa”	Anagram with itself, deduplication logic	Ensure frequency logic handles this
Spaces and punctuation	Palindrome checks that should ignore non-alphanumeric	Use Character.isLetterOrDigit() filter

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 38: Interview Practice: Strings

Practice Prompt: “Given two strings s and t, determine if t is an anagram of s.”

How to Walk Through This in an Interview:

Step 1: Understand (30 seconds)

“So I need to check if both strings contain exactly the same characters with the same frequencies. For example, ‘anagram’ and ‘nagaram’ would return true. An empty string is an anagram of another empty string. If the lengths differ, they cannot be anagrams.”

Step 2: Identify Pattern (15 seconds)

“This is a character frequency comparison problem. I have two approaches: sort both strings and compare, or use a frequency array.”

Step 3: Plan (30 seconds)

“I’ll mention the sort approach first. Sorting both strings and comparing takes O(n log n) time and O(n) space. But since the problem specifies lowercase English letters, I can use an int[26] frequency array for O(n) time and O(1) space. I’ll increment counts for the first string and decrement for the second, then check if all counts are zero.”

Step 4: Implement

```
public boolean isAnagram(String s, String t) {
    if (s.length() != t.length()) return false;

    int[] freq = new int[26];
    for (int i = 0; i < s.length(); i++) {
        freq[s.charAt(i) - 'a']++;
        freq[t.charAt(i) - 'a']--;
    }
    for (int count : freq) {
        if (count != 0) return false;
    }
```

```
        return true;
    }
```

Step 5: Verify

"Let me trace through s = 'anagram', t = 'nagaram'. After processing both strings, every character count nets to zero. The lengths match and frequencies match, so return true. Time is O(n), space is O(1)."

Common Interviewer Follow-ups:

Follow-up	How to Handle
"What if the input contains Unicode characters?"	"I'd switch from int[26] to a HashMap since the character set is no longer bounded."
"Can you do it with a single pass?"	"The solution above already processes both strings in a single loop by incrementing and decrementing simultaneously."
"What if you need to check millions of pairs against the same string?"	"I'd precompute the frequency array for the reference string once and compare against it for each candidate."

Common Fresher Mistakes on This Problem:

Jumping straight to HashMap when the problem says lowercase letters. Not checking length equality upfront (saves unnecessary work). Using two separate frequency arrays and comparing them element-by-element instead of using the single-array increment/decrement technique.

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 39: Section Cheat Sheet

String Operation Costs in Java:

Operation	Method	Time	Trap to Watch
Access char	charAt(i)	O(1)	None
Length	length()	O(1)	None
Substring	substring(i, j)	O(j-i)	Creates new String object
Concatenation	s1 + s2	O(n+m)	O(n^2) when used in a loop
Contains/indexOf	contains(t), indexOf(t)	O(n*m)	Hidden cost in loops
Equals	equals(other)	O(n)	Never use == for strings
StringBuilder append	append(c)	Amortized O(1)	Always use inside loops
toCharArray	toCharArray()	O(n)	Full copy, O(n) space

Character Frequency Approach Decision:

Constraint	Tool	Code Pattern
"lowercase English letters"	int[26]	freq[c - 'a']++
"ASCII characters"	int[128]	freq[c]++
"any character" / Unicode	HashMap<Character, Integer>	map.getOrDefault(c, 0) + 1
Digits only	int[10]	freq[c - '0']++

Pattern Recognition Signals:

Problem Signal	Pattern	Section
“Is this a palindrome?”	Two pointers, opposite direction	This section + Section 6
“Find longest palindromic substring”	Expand around center	This section
“Palindromic subsequence”	Dynamic Programming	C6
“Is t an anagram of s?”	Character frequency comparison	This section
“Group strings by anagram”	Sorted string as HashMap key	This section + Section 5
“Longest/shortest substring with condition”	Sliding window (variable size)	Section 7
“First unique character”	Frequency count then scan	This section + Section 5

Key Code Patterns:

```
// StringBuilder for building strings in loops
StringBuilder sb = new StringBuilder();

// toCharArray for in-place style work
char[] chars = s.toCharArray();

// int[26] frequency with c-'a' indexing
int[] freq = new int[26];
for (char c : s.toCharArray()) freq[c - 'a']++;

// Increment/decrement anagram check
for (int i = 0; i < s.length(); i++) {
    freq[s.charAt(i) - 'a']++;
    freq[t.charAt(i) - 'a']--;
}

// Expand around center template
int len = expandAroundCenter(s, i, i);       // odd
int len2 = expandAroundCenter(s, i, i + 1);  // even
```

Common Interview Mistakes:

Mistake	Why It Is Wrong	Better Approach
Using == for string comparison	Compares references, not content	Always use .equals()
String += in a loop	O(n^2) from repeated copying	Use StringBuilder
HashMap for lowercase-only input	Unnecessary overhead, misses constraint	int[26] array
Substring techniques on subsequence problems	Subsequences can skip characters	DP required (C6)
Not checking length equality for anagrams	Wastes time processing strings that cannot match	Early return if lengths differ
Initializing palindrome check without edge case	Empty or single-char strings need handling	Return true for length 0 or 1

How This Connects to Other Sections:

This Section's Concept	Connects To	How
Two-pointer palindrome check	Section 6: Two Pointers	Same opposite-direction template, applied to characters
Character frequency maps	Section 5: Hash Maps	Frequency counting is a specialized hash map application
Longest substring without repeating	Section 7: Sliding Window	Same problem, formalized with the variable-size window template
String concatenation cost	Section 2: Complexity	Hidden O(n) body inside O(n) loop creates O(n^2), same concept as hidden loop costs
Sorted string as HashMap key	Section 5: Hash Maps, Section 9: Sorting	Sorting as preprocessing to enable grouping

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina